

Modern Web Frontend

IF3110 Web-Based Application Development
School of Electrical Engineering and Informatics
Institut Teknologi Bandung

Web Frontend Evolution

- **1990-1991** – The first website is published, made of simple static HTML pages.
- **1995-1996** – JavaScript and CSS are introduced, adding interactivity and styling.
- **2005** – Ajax lets pages load data in the background without a full reload.
- **2006** – jQuery makes JavaScript easier and works the same across browsers.
- **2007** – The iPhone launches, and browsing the web on mobile becomes mainstream.
- **2010** – Responsive Web Design is introduced, and early SPA frameworks like AngularJS appear.
- **2013** – React is open-sourced, helping popularize declarative, component-based UI development.
- **2014** – HTML5 becomes a W3C Recommendation, and Vue.js is publicly released.
- **2015** – Progressive Web Apps (PWA) are introduced, bringing app-like features to websites.
- **2016** – Next.js popularizes server-side rendering, and the micro-frontend concept emerges.
- **2023-now** – Hybrid rendering, server components, and AI-assisted development shape the modern frontend.

Modern Web Frontend: What's Different?

- User experience is now the central focus, with websites expected to be fast, smooth, and easy to use.
- Websites have shifted from static pages into interactive, app-like experiences.
- A single website adapts its layout to phones, tablets, and large screens.
- Interfaces are built from reusable components instead of manually manipulating the DOM.
- Websites can be installed like apps and keep working without an internet connection.
- The web is designed to be inclusive for everyone, including people with disabilities and users from any country, language, or culture.

Modern Web Frontend Technique

- Responsive Web Design (RWD)
- Single Page Application (SPA)
- Server Side Rendering (SSR)
- Web Accessibility
- Progressive Web App (PWA)
- Internationalization (i18n)
- Localization (l10n)
- Real-time Web Apps (Websocket, SSE)
- WebAssembly
- WebAuthn

Responsive Web Design

Responsive Web Design

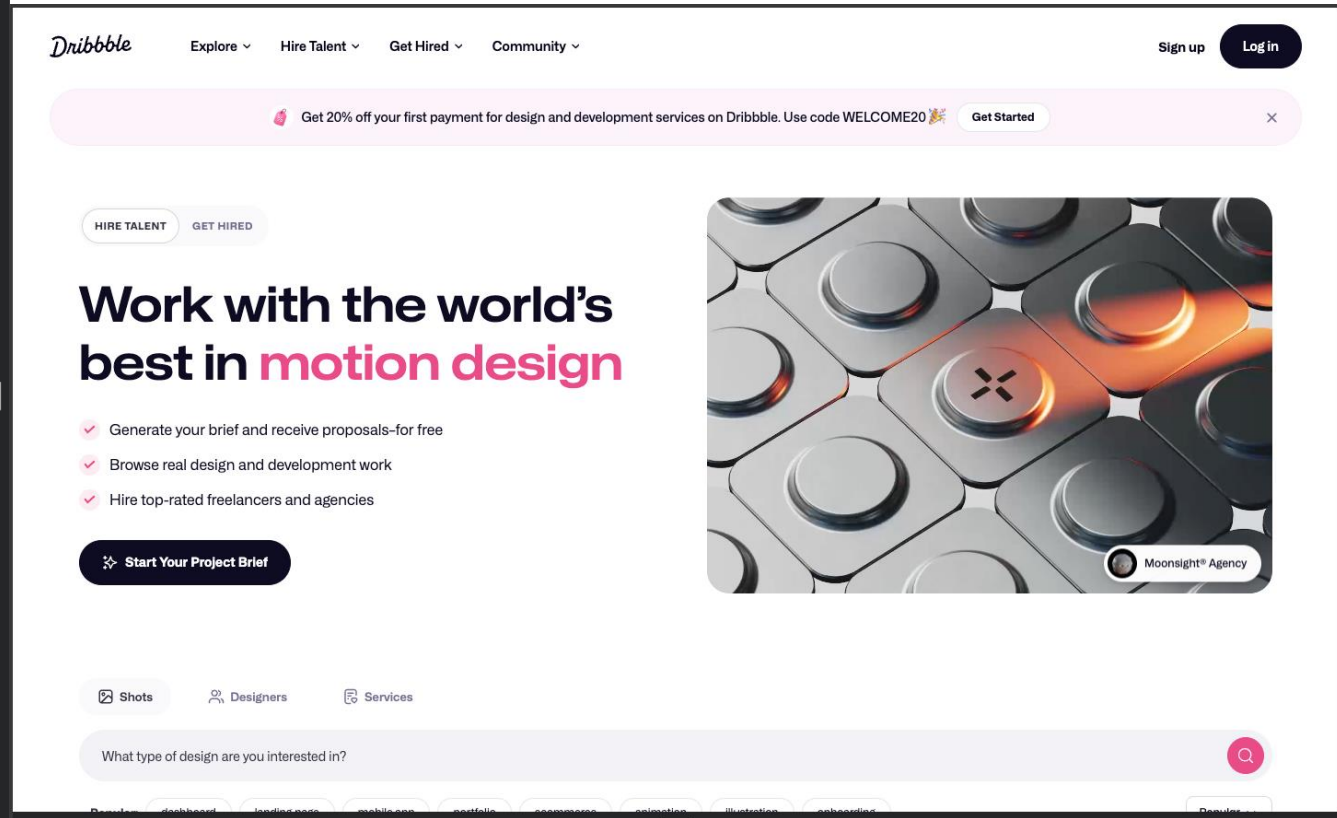
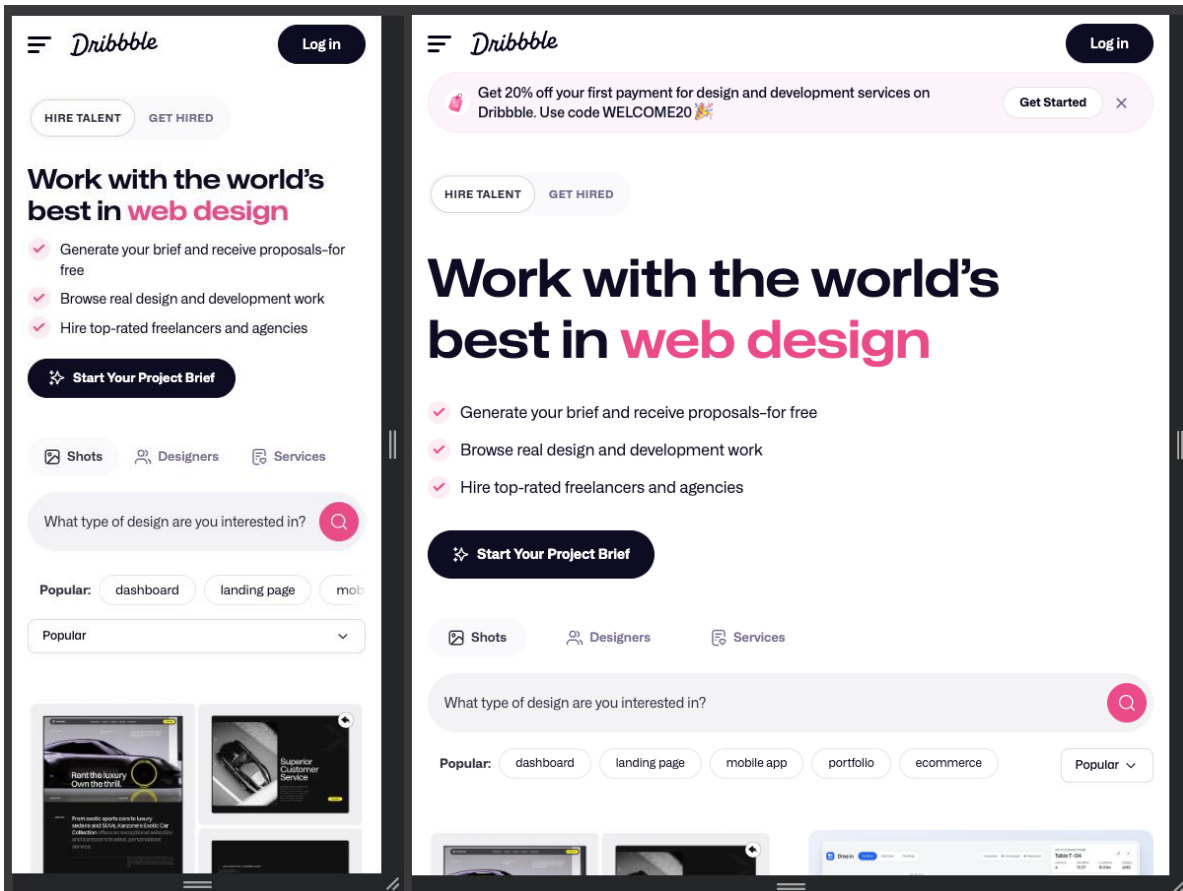
RWD is the concept of developing a website in a way that **allows the layout to adjust** according to the user's screen resolution (view port) using media queries



Why do you need a responsive web app?

- Growing demand for smartphones
- Multiple screen sizes and mobile browsers
- Permits wider browser support
- Compulsory for getting good business

Example: Dribbble



How do we design for RWD

Use the ***Mobile First Approach*** and favor ***Progressive Enhancement*** instead of the traditional ***Graceful Degradation***

Understanding the Mobile Web and its Content

Web designers should consider the following questions while mobilizing the content:

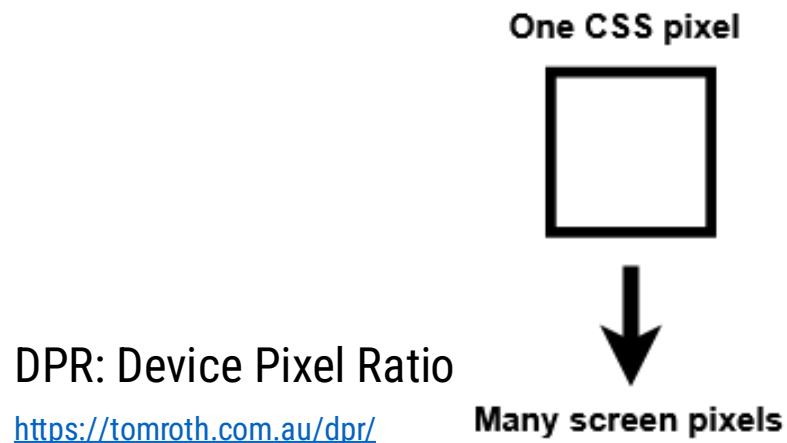
- Q. What is the purpose of the site?
- Q. Who are the users?
- Q. What sort of data is being accessed by them?
- Q. User's locations ?
- Q. What's the capacity of the devices and screen information such as size, resolution, pixel density, and color information from which information is being accessed?

Mobile First Perspective Content

- A responsive website does not make responsive content
- Curate your content down to its essential points
- Consider the needs of the mobile user first

Viewport

A viewport is the visible area of a web page on a device screen, excluding browser menus and toolbars



DPR: 1



DPR: 2



DPR: 2.25



DPR: 3

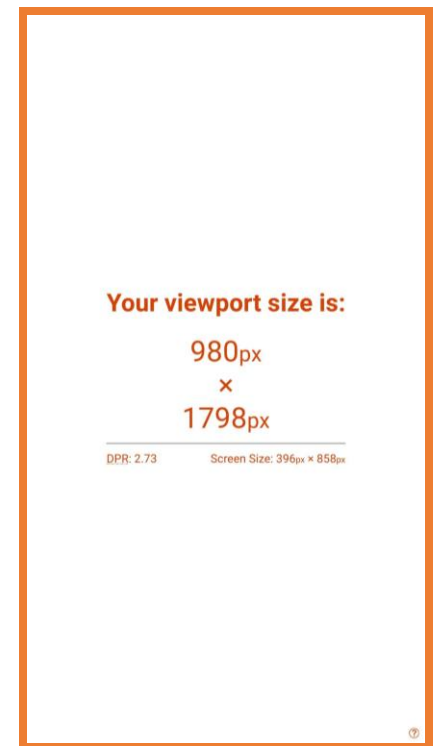


CSS width = 396



Viewport enabled
(mobile mode)

CSS width = 980



Desktop mode

Viewport Controlling



Without viewport meta tag



With viewport meta tag

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta name="viewport" content="width=device-width, initial-scale=1.
5 </style>
6 img {
7     max-width: 100%;
8     height: auto;
9 }
10 </style>
11 </head>
12 <body>
13 <p><b>To understand this example, you should open this page on a ph
14
15 
16
17 <p>
18 Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam
19 Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorpe
20 Nam liber tempor cum soluta nobis eleifend option congue nihil impe
21 </p>
22
23 </body>
24 </html>
```

https://www.w3schools.com/css/css_rwd_viewport.asp

Strategies to implement a responsive design

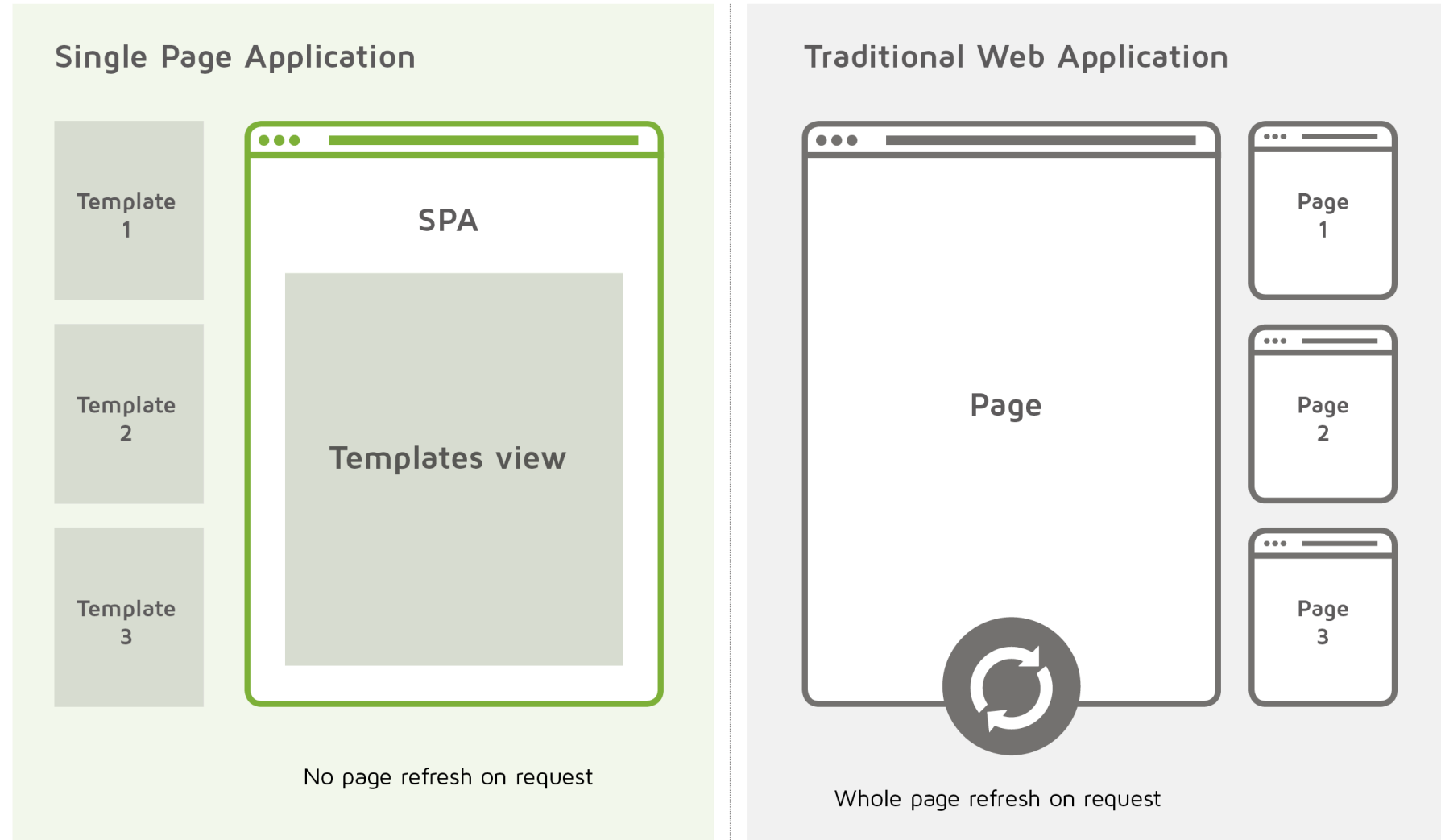
1. CSS Media Queries
 - W3C Recommendation, 19/06/2012
 - A simple example: https://www.w3schools.com/css/css3_mediaqueries_ex.asp
 - Typical implementation: a responsive grid system
 - <https://getbootstrap.com/docs/3.4/css/#grid>
2. CSS Flex Box
 - W3C Candidate Recommendation, 19/11/2018
 - A complete guide: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
 - Example: <https://alistapart.com/>
3. CSS Grid (not to be confused with bootstrap grid system above)
 - ⑩ W3C Candidate Recommendation, 18/12/2020
 - ⑩ Flex is one dimensional (horizontal/vertical) while Grid is two dimensional.
 - ⑩ Complete guide: <https://css-tricks.com/snippets/css/complete-guide-grid/>

Single Page Application (SPA)

Single Page Application

- A **single-page application (SPA)** is a web application or website that interacts with the user by dynamically rewriting the current web page with new data from the web server, instead of the default method of a web browser loading entire new pages.
- The goal is faster transitions that make the website feel more like a native app.
- Example: Google Workspace (Gmail, Drive, Docs, etc), Facebook, Instagram

SPA vs. Traditional Page



SPA vs. Traditional Page

Aspect	Traditional (Multi Page Application)	Client-rendered (Single page Application)
Navigation response	New HTML document	Often data for the next view
UI construction	Commonly server templates	Browser application code
Routing	Server resolves document paths	Client router resolves view paths
Navigation behavior	Document replacement	View update within the document
State lifetime	Page memory resets on navigation	Application memory persists
API separation	Optional	Common

Advantages of SPA

- Quick, majority of resources (HTML+CSS+Scripts) only loaded once
- Development is usually quicker and more efficient
- Optimised for debugging, developers can monitor network operations, investigate page elements and data associated
- Easier to create a mobile application based on it, developers can reuse the same backend code
- Efficient in caching any local storage, even offline

Disadvantages of SPA

- Need special tricks for SEO
- Slow to download, heavy client frameworks are required to be loaded to the client
- Require JavaScript in its active mode in users' browsers
- Less secure due to Cross-Site Scripting (XSS), it enables attackers to inject client-side scripts
- Memory leaks in JavaScript can lead to productivity drop for even powerful systems

Technical/Pattern approaches

- **Document Hashes (URI fragments)**
HTML/CSS authors can leverage element IDs or #target to show or hide different sections of the HTML document.
- **History API (new approach, more SEO friendly)**
The History API provides access to the browser's session history. It exposes useful methods and properties that let you navigate back and forth through the user's history and manipulate the contents of the history stack.
- **AJAX**
AJAX using asynchronous requests to a server for XML or JSON data using XMLHttpRequest or Fetch API.
- **WebSockets**
bidirectional real-time client-server communication technology
- **Server-sent events**
servers can initiate data transmission to browser clients.
- **Server-Side Rendering**
Rendering initial HTML on server for performance and SEO, then hydrate on client.
- **Micro-frontend Architecture**
Split large apps into independently deployable frontend module
- **Code splitting and Lazy Loading**
Load routes/components on demand

Rendering Strategy

- Client-Side Rendering (CSR)
- Server-Side Rendering (SSR)
- Static Site Generation (SSG)
- Hybrid Rendering Approaches
 - Incremental Static Regeneration (ISR)

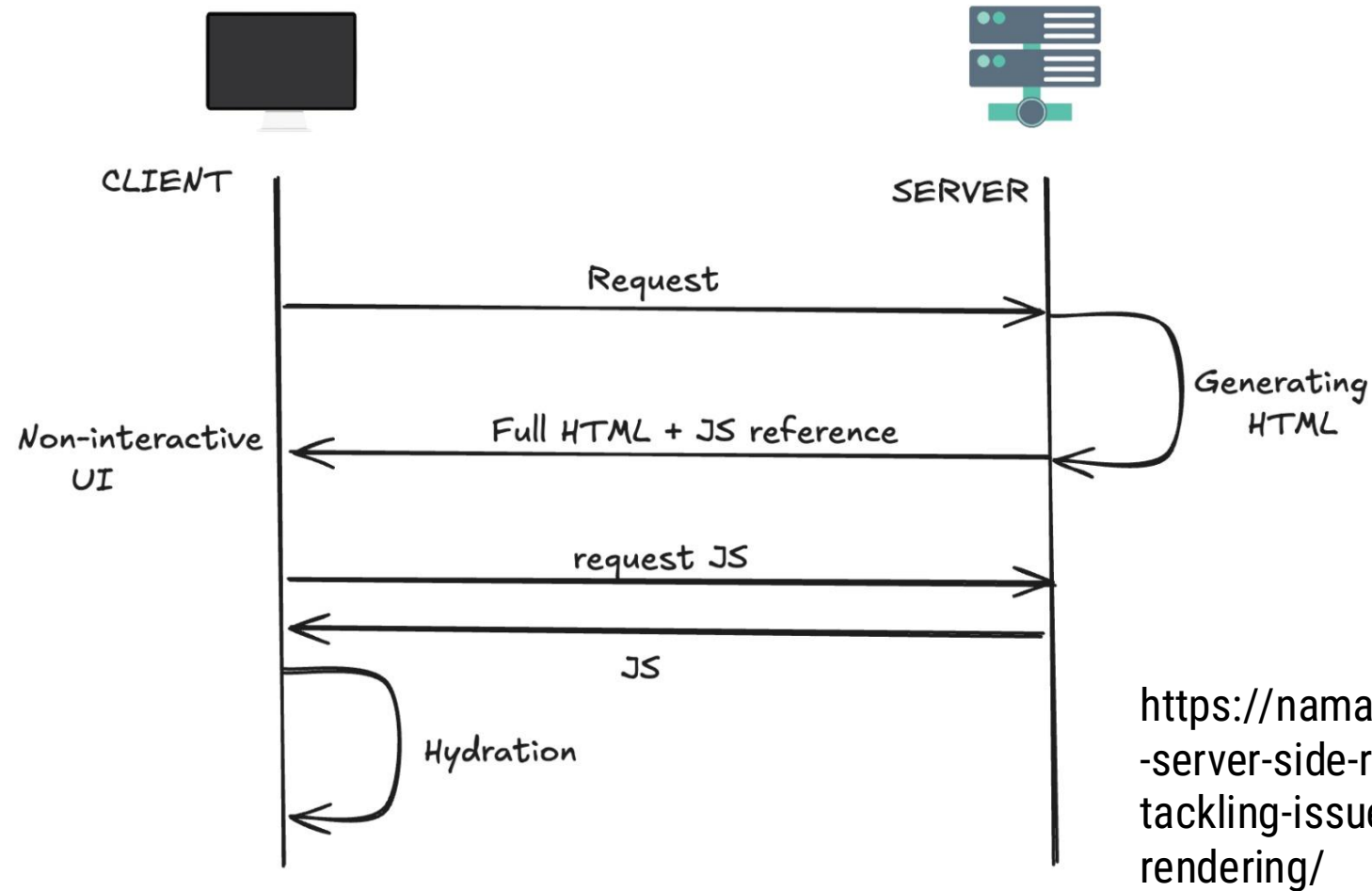
Client Side Rendering (CSR)

- **Client-side rendering (CSR)** is an approach to building web pages where the browser, not the server, builds most of the page using JavaScript.
- Server sends a nearly empty HTML file and a JS file, then client renders the page

Server Side Rendering (SSR)

- CSR is not SEO-friendly (also not LLM-friendly). All content in CSR will be loaded after browser loads the page.
- SSR builds complete HTML on the server for every request, so users and search engines see content immediately.
- With SSR, the server runs your components, fetches data, and sends finished HTML.

How SSR works



<https://namastedev.com/blog/why-server-side-rendering-next-js-tackling-issues-of-client-side-rendering/>

Hydration

- **Hydration** is the process of making server-rendered HTML interactive. The browser displays the HTML first, then JavaScript connects it to the app's behavior.
- Hydration Step:
 - **DOM Matching:** The framework matches the existing HTML with the app's expected structure so it can reuse those elements.
 - **Event Listener Attachment:** It connects handlers for actions such as clicks, typing, and form submissions.
 - **State Initialization:** It initializes the app's state, often using data embedded in the HTML during the server render.
 - **Reconciliation:** Framework checks for differences between the server-rendered output and the Virtual DOM and resolves them as needed.

Hydration Mismatch

- **Hydration mismatch** happens when the server-rendered HTML differs from what the client expects during its first render.
- Common Causes:
 - **Nondeterministic Values:** `Date.now()`, `new Date()`, `Math.random()`, or generated IDs that differ between renders.
 - **Browser-only APIs during Render:** reading `window`, `localStorage`, `navigator`, or screen size during render.
 - **Invalid HTML nesting:** for example, a `<div>` inside a `<p>`, or a `<p>` inside another `<p>`. The browser's parser "fixes" the HTML, so the DOM no longer matches what React expects.
 - **Locale and timezone differences:** `toLocaleDateString()` formats differently when the server runs in UTC and the user is in, say, WIB (UTC+7).

Hydration Mismatch (2)

- Depending on the framework and version, a mismatch can cause:
 - **Framework throwing away the server HTML and re-rendering the whole tree on the client**, which hurts performance and loses the SSR benefit
 - Console warnings or errors
 - Visible flicker or layout shift
 - Event handlers attached to the wrong elements in subtle cases.

Benefits of SSR

- **Accessibility:** Content is readable without JavaScript, but interactive features may not work.
- **Crawler-friendliness:** Search engines and social media bots can read content without running JavaScript.
- **Performance:** The server can fetch data upfront, reducing delays from sequential browser requests.

Drawbacks of SSR

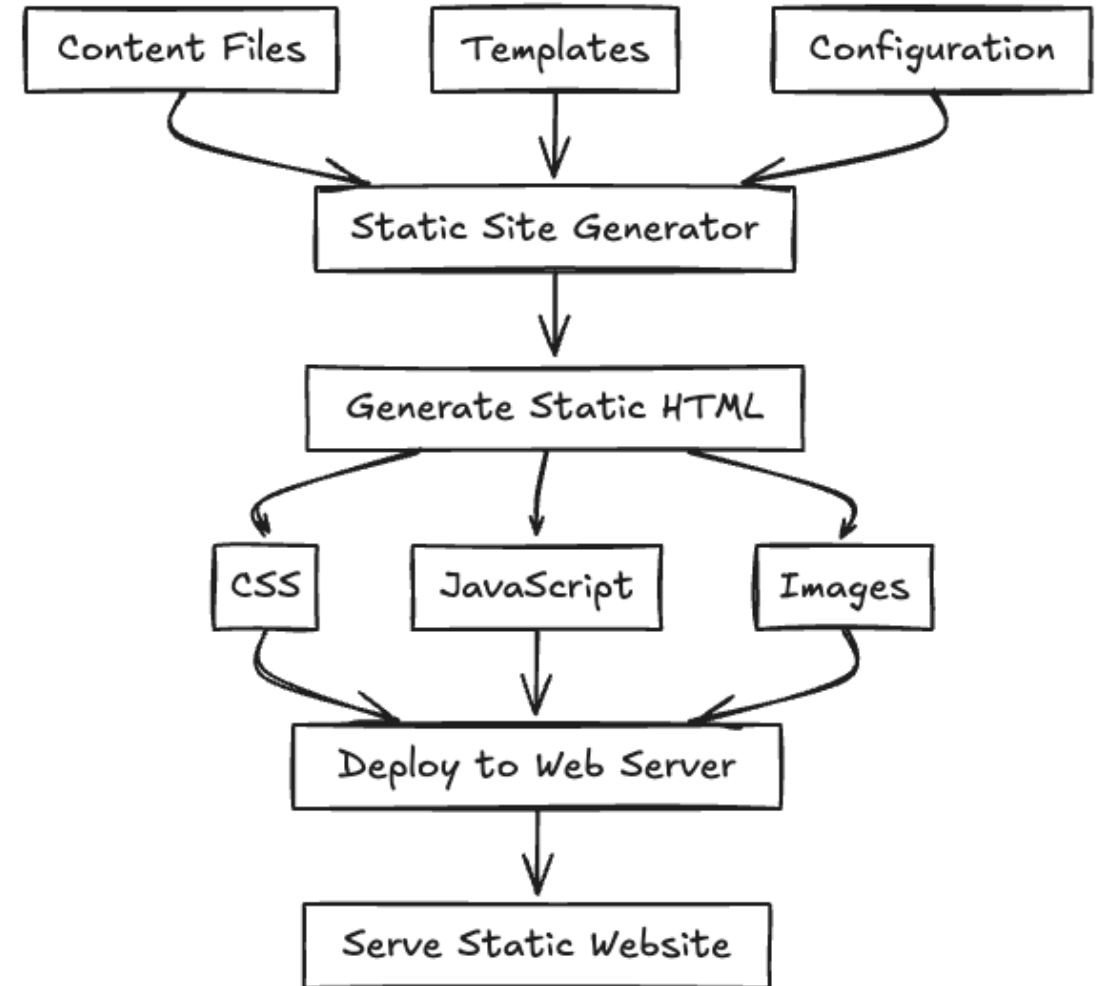
- **Server cost and scaling:** every visit runs code and often queries a database. Traffic spikes need more servers.
- **Slower Time to First Byte:** the server must finish fetching and rendering before sending the first byte.
- **Hydration cost:** the page looks ready before it is, and users may click buttons that don't respond yet.
- **Complex caching:** Caching dynamic SSR pages on a Content Delivery Network (CDN) is harder than caching static files.

Static Site Generation (SSG)

- SSR generates the HTML on the server for every request. That keeps content always fresh, but it has Higher server load and cost.
- **SSG (Static Site Generation)** is a way of building websites where every page is rendered into a finished HTML file **at build time**, before anyone visits the site.
- Those files are then served as-is from a web server or CDN.

How does SSG work?

- A static site generator takes your content (Markdown files, data from a CMS or API, JSON) and your templates or components, combines them, and writes out a folder of plain HTML, CSS, and JavaScript.
- Those files are then served as-is from a web server or CDN.



<https://dev.to/lakshmananarumugam/ssr-ssg-a-game-changer-for-your-web-app-32e9>

Noteworthy

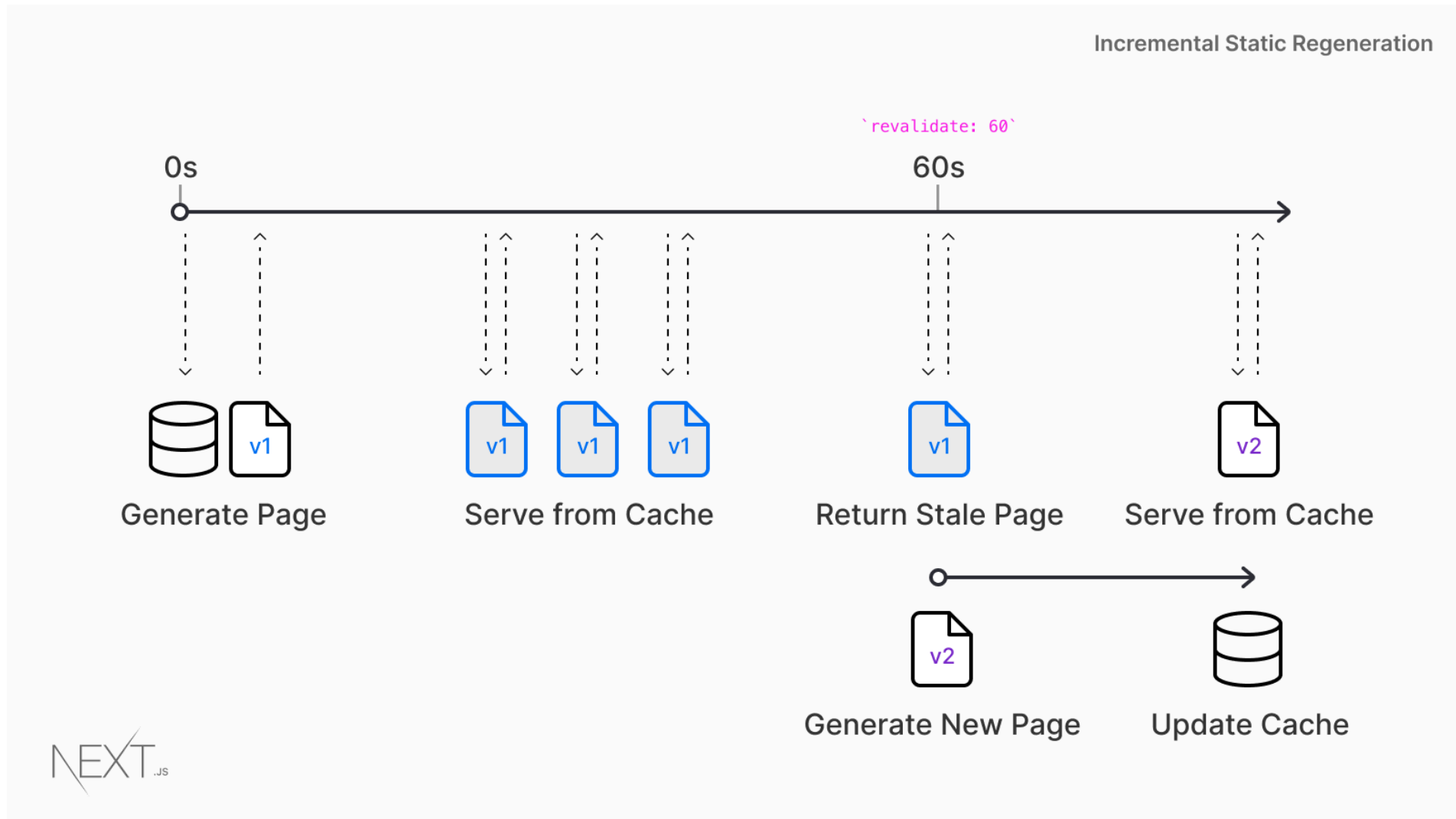
Here are the key things to be aware of in SSG:

- **Stale content.** Pages only show data from the last build, so changes in your CMS or database won't appear until you rebuild. ISR can help you to address this issue.
- **Long build times.** Every page is built in advance, so large sites can take a long time to build. Pre-build only important pages and generate the rest on demand.
- **Cache issues.** CDNs and browsers may serve old versions after a deploy. Use short cache times for HTML and hashed filenames for JS/CSS.

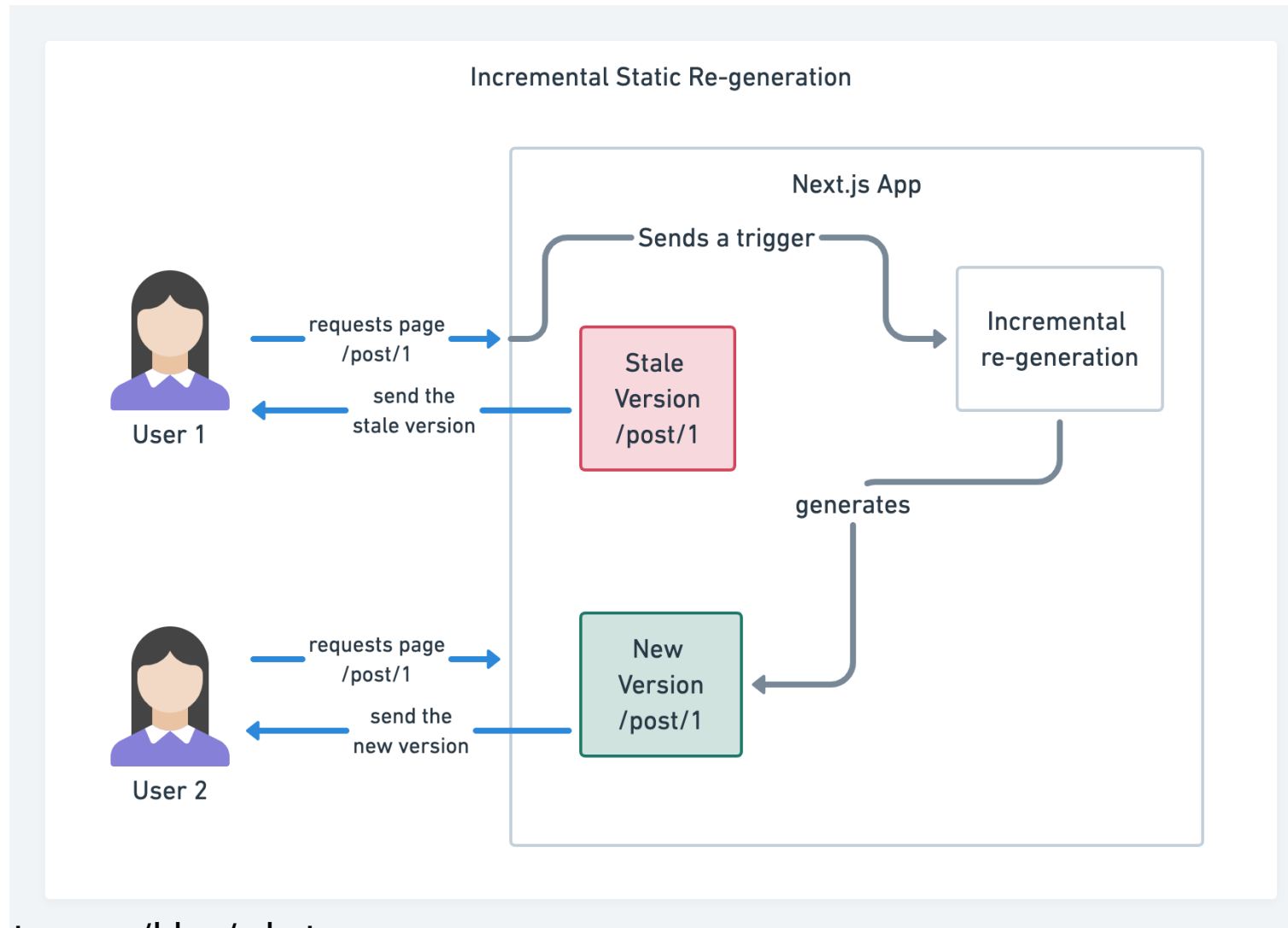
Incremental Static Regeneration (ISR)

- SSG content becomes stale until you rebuild, and big sites take a long time to build. SSR keeps content fresh but costs server work on every request.
- **ISR (Incremental Static Regeneration)** is a hybrid between SSG and SSR.
- Pages are generated as static HTML like SSG, but they can be **updated in the background after deployment** without rebuilding the whole site.
- It was popularized by Next.js, and similar features exist in Nuxt, SvelteKit, and Astro (with adapters).

How does ISR work?



How does ISR work?



<https://websolutionmaster.com/blog/what-are-incremental-static-regeneration>

Noteworthy

Here are the key things to be aware of in ISR:

- **The first visitor after expiry sees old content.** Regeneration happens in the background, so the request that triggers it still gets the stale page. Don't use ISR where every visitor must see the latest data.
- **Debugging can be confusing.** It's hard to tell which version a user saw.
- **Failed regeneration is silent.** If your API errors during regeneration, the old page keeps being served with no visible error. Add logging and monitoring so you notice when pages stop updating.

Comparison

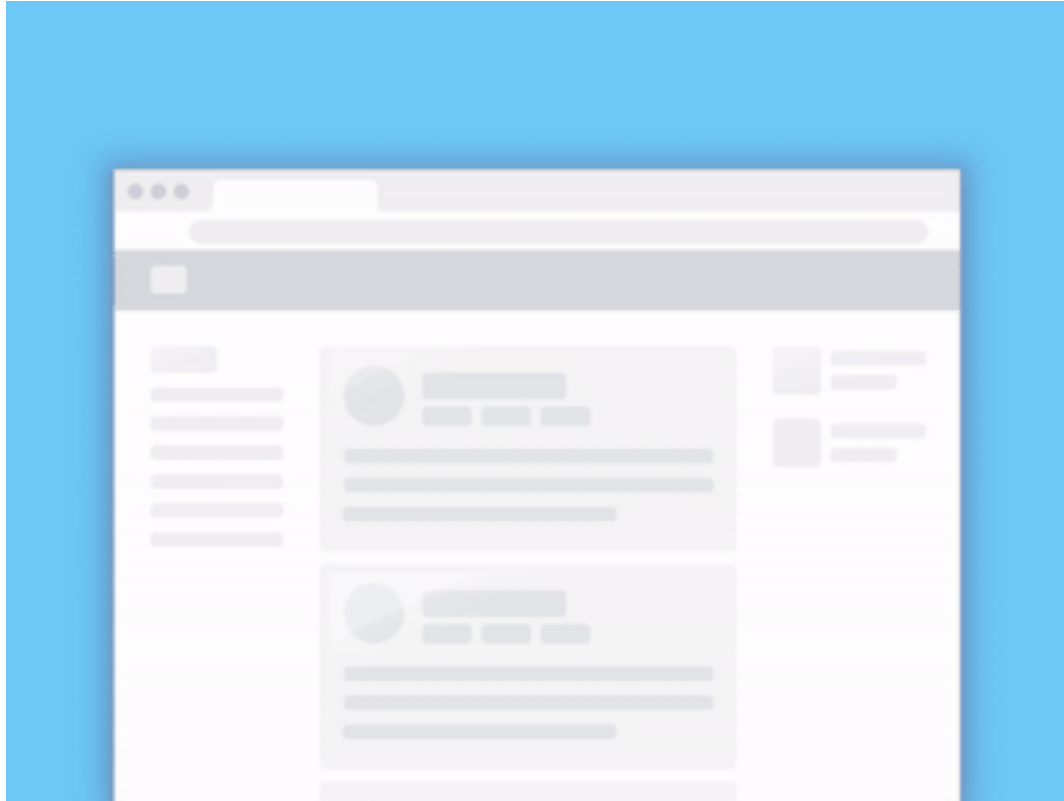
Aspect	CSR (Client Side Rendering)	SSR (Server Side Rendering)	SSG (Static Site Generation)	ISR (Incremental Static Regeneration)
When HTML is generated	In the browser, at runtime	On the server, per request	At build time	At build time, then re-generated periodically or on demand
Navigation after load	Very fast, app-like	Fast with client-side routing after hydration	Fast with client-side routing	Fast with client-side routing
Data freshness	Always fresh (fetched client-side)	Always fresh (fetched per request)	Stale until next rebuild	Fresh within the revalidation window
SEO	Weak by default (crawlers may see empty HTML)	Excellent	Excellent	Excellent
Server cost/load	Low (static hosting + API)	Highest (compute on every request)	Lowest (just a CDN)	Low (occasional regeneration)
Scalability	High	Needs scaling of server capacity	Very high	Very high
Build time	Short	Short	Grows with page count	Short (can build only key pages, rest on first request)
Personalization per user	Easy	Easy	Hard (needs client-side fetching)	Hard (same as SSG)
Complexity	Low-moderate	Moderate-high (hydration, caching, server errors)	Low	Moderate (cache invalidation logic)
Time to First Byte (TTFB)	Fast (but content arrives late)	Slower	Very fast	Very fast

Best Practice

Best Practice

- **Split code by route.** Load each page's code only when it is visited.
- **Avoid importing every page at the top of App.jsx.** It forces users to download the whole app to see one page. **Use Lazy import.**
- **Handle Empty case or Error Properly.** Always handle loading, error, and empty cases.
- **Watchout with your secret!** Anything passed to a client component is visible in the page source
- **Show skeletons, not blank screens.** A skeleton that matches the final layout makes waiting feel shorter and avoids layout shift.

Skeleton Loading & Empty State Handling



<https://dribbble.com/shots/5460452-Skeleton-Loading-Screen>



<https://dribbble.com/shots/6305886-Empty-Cart-Empty-State>

Rendering Strategy Best Practice

- Choose suitable rendering strategy.
- When you are using SSR/SSG/ISR, **Send only what the client needs.**
- When you are using SSR/SSG/ISR, **Render client-only values after mounting.**

Rendering Strategies

Still Confused? Here's a Simple Rule of Thumb:

SSG  If you want a blazing-fast page that rarely changes	CSR  If you want interactivity without worrying about SEO
SSR  If you want the most up-to-date data on every visit	ISR  If you want fast pages that update behind the scenes

<https://medium.com/@rggamage1997/understanding-ssr-ssg-isr-and-hybrid-rendering-in-next-js-real-world-examples-and-use-cases-fd6431c33fbd>

Framework

